

Deep Learning - Practical Work

1st Apinaja Ananthakumar

ISEP

Porto, Portugal

1252373@isep.ipp.pt

2nd Agata Szysiak

ISEP

Porto, Portugal

1252365@isep.ipp.pt

Abstract—This paper presents the development and evaluation of deep learning models for image classification and time series forecasting. In the image classification task, convolutional neural networks (CNNs) were applied to the Fruits-360 dataset. Various architectures were tested to analyze the impact of filters, pooling layers, and data augmentation on classification performance, including custom designs and a pre-trained MobileNetV2. For time series forecasting, recurrent neural networks (RNNs) were used to predict temperatures 24 hours into the future using the Jena Climate dataset. The performance of long short-term memory (LSTM) networks was compared against SimpleRNN layers across different historical time windows. The results demonstrate the efficiency of transfer learning for visual tasks and highlight the stability of LSTMs over traditional RNNs when processing longer temporal sequences.

Index Terms—Convolutional neural networks (CNNs), data augmentation, image classification, long short-term memory (LSTM), recurrent neural networks (RNNs), time series forecasting, transfer learning.

I. INTRODUCTION

Deep learning has become a fundamental tool in solving complex real-world problems involving images, text, and time series data [1]. This paper documents the practical work developed in the scope of the Deep Learning course, with the objective of building and evaluating neural network models using TensorFlow and Keras [2].

Two problems are addressed. The first involves image classification using Convolutional Neural Networks (CNNs) applied to the Fruits-360 dataset, with the goal of correctly identifying 260 fruit and vegetable categories. The second involves time series forecasting using Recurrent Neural Networks (RNNs) applied to the Jena Climate dataset, with the goal of predicting air temperature 24 hours ahead based on historical meteorological observations.

For each problem, multiple model architectures and hyperparameter configurations were systematically explored and compared. The results, learning curves, and individual predictions are analyzed and discussed to draw conclusions about the effectiveness of each approach.

All code and supplementary materials are available at [3].

II. CNN MODEL FOR FRUIT CLASSIFICATION

A. Dataset and Preprocessing

The Fruits-360 dataset [4] contains images of various fruits and vegetables. The dataset comprises a total of 260 different classes. All images are pre-segmented and standardized, and were captured against a white background. This clean data

structure enables faster convergence during CNN training and provides an ideal foundation for experiments with different model architectures. The following preprocessing steps were applied to prepare the data for training:

1) *Image Resizing*: A uniform target size of 100x100 pixels was established for all images to guarantee a consistent input matrix for the convolutional neural network, resulting in a final input shape of 100x100x3 since the images are RGB color images.

2) *Normalization*: Since deep neural networks are sensitive to unscaled and large input values, the pixel values of the images were normalized. The original color values, ranging from 0 to 255, were scaled down using the $\text{rescale}=1./255$ function in the ImageDataGenerator to a continuous range of [0, 1]. This normalization ensures that the gradients remain stable during training and that the model learns more efficiently.

3) *Data Splitting*: The dataset was provided as separate training and test directories. The test directory contained 45,724 images, while the training directory was further divided using an 80:20 stratified split to obtain a validation set dedicated to hyperparameter monitoring and early stopping. This split resulted in a total of 182,945 images, which are distributed across 109,865 training images, 27,356 validation images, and 45,724 test images.

4) *Data Batching and Loading*: Keras' ImageDataGenerator [2] was used to load the images directly from the directories in randomly shuffled batches of 32. This batch-wise loading enables memory-efficient processing, since the full dataset of 182,945 images does not need to be held in memory at once, while the random shuffling ensures robust gradient estimation at each training step.

B. Methodology

1) *Baseline Architecture*: CNNs were used to address the fruit classification task due to their high efficacy in automatically extracting spatial hierarchies of features from images. The baseline architecture was designed with three sequential convolutional blocks using 32, 64, and 128 filters with 3x3 kernels. Each block was followed by a batch normalization layer [5] to stabilize and accelerate the learning process and a max pooling layer to reduce the spatial dimensions and extract the most prominent local features. The network concludes with a fully connected classification head comprising a dense layer with 256 units, a dropout layer [6] with a rate of 0.5

for regularization, and a Softmax output layer that outputs probabilities for the 260 distinct fruit categories.

2) *Hyperparameter Optimization Process*: The optimization process involved systematically exploring architectural modifications rather than conducting random grid searches. Experiments included scaling the network’s capacity by varying the number of convolutional filters per block. Specifically, a lightweight “Small CNN” with 16, 32, and 64 filters and a high-capacity “Large CNN” with 64, 128, and 256 filters were tested against the baseline. Further architectural tuning involved comparing MaxPooling and AveragePooling, as well as altering the stride (2x2) and padding (“valid”) parameters, to observe their impact on spatial downsampling and computational efficiency. For the Large CNN, the learning rate was additionally reduced to 0.0001 to ensure stable convergence given its increased capacity.

3) *Training Configuration and Metrics*: The models were compiled using the Adam optimizer [7], chosen for its adaptive learning rate and robust efficiency in handling the complex parameter spaces associated with deep learning models. Since the objective is a multi-class classification problem involving 260 mutually exclusive classes, categorical cross-entropy was selected as the loss function. Model performance was primarily evaluated using accuracy, which provides a direct and intuitive interpretation of overall correctness. The macro-averaged precision, recall, and F1-score were tracked to provide a comprehensive model evaluation. This approach addresses the problem’s specific constraints by ensuring consistent performance across all classes. Consequently, the model’s robustness can be verified, even when there are visually similar subcategories.

4) *Model Validation Approach*: The network’s generalization capabilities were continuously monitored using a 20% hold-out validation set derived from the training data. An early stopping mechanism with a patience level of five epochs was implemented during training. This measure prevents overfitting, a common issue in high-capacity CNNs. Training was automatically halted if the validation loss did not improve for five consecutive epochs to avoid overparameterization. Additionally, a model checkpoint was employed to ensure the final model used the optimal weights from the best-performing epoch instead of the weights from the interrupted final epoch.

C. Model Architecture

Among the tested configurations, the custom “Large CNN” performed best with a test accuracy of 97%. Meanwhile, MobileNetV2 led slightly in the overall macro-averaged metrics with 98%. The network processes 100x100x3 RGB input images through a series of layers designed to progressively extract spatial features. Feature extraction relies on three sequential convolutional blocks that use 3x3 filters, zero padding (padding=“same”), and ReLU activation. The filter depth increases from 64 to 128 and finally to 256 across the blocks. After each convolution, batch normalization is applied to stabilize the learning process. Each block concludes with a 2x2 max-pooling layer. This downsampling step reduces the spatial dimensions while controlling parameter growth and

preserving prominent local features, such as fruit edges and shapes. After feature extraction, the resulting 3D tensor is flattened into a 1D vector and fed into a fully connected dense layer with 256 units and ReLU activation. A dropout rate of 0.5 is applied to mitigate overfitting in this high-capacity section. The network concludes with a dense output layer of 260 units that uses a Softmax activation function to generate mutually exclusive class probabilities for the 260 fruit categories. Table I summarizes the complete structure, and Fig. 1 provides a visual representation.

TABLE I
MODEL SUMMARY OF THE BEST CUSTOM CNN ARCHITECTURE (LARGE CNN, 100x100x3 INPUT).

Layer (Type)	Output Shape	Param #
Conv2D (64, 3x3, same)	(None, 100, 100, 64)	1,792
BatchNormalization	(None, 100, 100, 64)	256
MaxPooling2D (2x2)	(None, 50, 50, 64)	0
Conv2D (128, 3x3, same)	(None, 50, 50, 128)	73,856
BatchNormalization	(None, 50, 50, 128)	512
MaxPooling2D (2x2)	(None, 25, 25, 128)	0
Conv2D (256, 3x3, same)	(None, 25, 25, 256)	295,168
BatchNormalization	(None, 25, 25, 256)	1,024
MaxPooling2D (2x2)	(None, 12, 12, 256)	0
Flatten	(None, 36864)	0
Dense (256, ReLU)	(None, 256)	9,437,440
Dropout (0.5)	(None, 256)	0
Dense (260, Softmax)	(None, 260)	66,820
Total params	9,876,868	
Trainable params	9,875,972	
Non-trainable params	896	

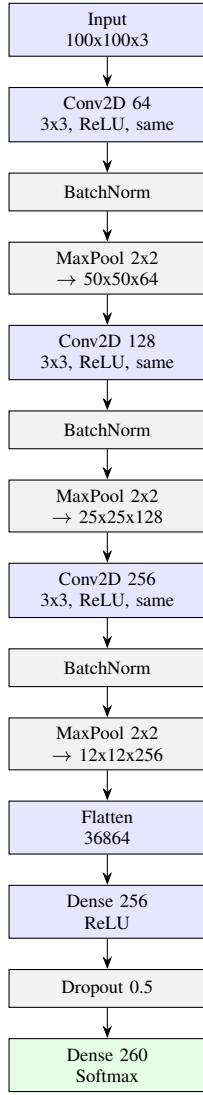


Fig. 1. Architecture of the best custom CNN (Large CNN).

D. Impact of Filters, Strides, Padding, and Pooling

Several variations of the baseline model were analyzed to assess the influence of different architectural choices. Reducing the number of filters in the "Small CNN" (from 16 to 64) surprisingly yielded a highly competitive test accuracy of 97%. This shows that, due to the standardized and pre-segmented nature of the dataset, a lightweight model can effectively classify images with fewer parameters. However, scaling up the network capacity in the "Large CNN" (64, 128, and 256 filters) produced the most robust overall performance, achieving a test accuracy of 97% and the highest macro-averaged precision, recall, and F1 score (all 0.97). Replacing MaxPooling with AveragePooling in the pooling mechanism resulted in lower performance, yielding a test accuracy of 94%. These results suggest that MaxPooling is significantly more effective for this computer vision task. MaxPooling preserves prominent local features, such as sharp edges and distinct shapes of fruits, whereas AveragePooling dilutes these strong activations by calculating local averages. Finally, we tested

the impact of spatial downsampling by applying a 2x2 stride and "valid" padding to the first convolutional layer instead of the default 1x1 stride and "same" padding. This configuration reduced computational time per epoch by shrinking the spatial dimensions of the feature maps earlier in the network. Although this configuration achieved a respectable test accuracy of 96%, the validation loss curve exhibited greater instability during training than the baseline, suggesting that aggressive early downsampling, combined with the loss of border pixels, sacrifices valuable spatial information.

E. Convolution Visualization

For a better understanding of the internal feature extraction mechanisms of the CNN, we visualized the activation maps of the first convolutional layer of the most effective Large CNN model. An input image from the test set belonging to the "Almonds 1" class was fed into the network, and the corresponding outputs of the initial filters were plotted. As shown in Fig. 2, the early convolutional layer primarily extracts low-level features. The visualized feature maps highlight the almond's outer edges, contours, and high-contrast boundaries against the standardized white background. While some filters focus on horizontal or vertical edge detection, others activate in response to the almond's specific textures and color gradients. This visualization confirms that, in its initial stages, the network successfully learns to isolate fundamental structural characteristics, which are subsequently combined into more complex and abstract representations in deeper layers.

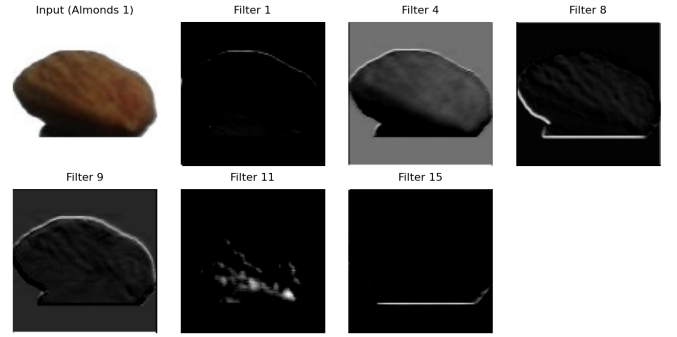


Fig. 2. Input image (*Almonds 1*) and feature maps from selected filters of the first convolutional layer.

F. Data Augmentation

To evaluate the impact of expanding the artificial dataset on the model's generalization capabilities, mild data augmentation techniques were applied to the training set. Specifically, random rotations (up to 10°), horizontal and vertical shifts (5%), and zooming (10%) were introduced. When the model trained with data augmentation was compared to the baseline CNN, the overall test accuracy remained highly competitive at 95%. The augmentation process acted as a subtle regularizer, leading to a slight improvement in macro-averaged precision (0.96) compared to the baseline (0.94). Since Fruits-360 images are

already centered, pre-segmented, and captured against a clean white background, heavy spatial transformations are unnecessary and could hinder initial convergence. Thus, mild data augmentation provided a marginal but measurable improvement in distinguishing visually similar subcategories without compromising the model’s fundamental accuracy.

G. Overfitting Analysis

Overfitting is a common challenge in deep neural networks [1]. It occurs when a model perfectly memorizes the training data but cannot generalize to new data. An analysis of the learning curves for the Large CNN model with high capacity (Fig. 3) clearly shows overfitting beginning in later epochs. By epoch 12, training accuracy climbed to an exceptional 99.43%, while validation accuracy plateaued at around 96.51%. Concurrently, the validation loss began to trend slightly upward, diverging from the continuously decreasing training loss.

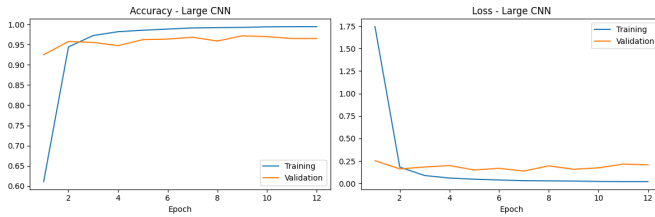


Fig. 3. Training and validation accuracy/loss curves for the Large CNN model, illustrating the onset of overfitting.

With the aim of combating this and preventing the network from memorizing noise, two solutions were implemented. First, a dropout layer with a rate of 0.5 was integrated into the fully connected classification head to force the network to learn redundant and robust feature representations [6]. Second, an early stopping callback was applied to monitor the validation loss. This mechanism successfully detected the divergence between the two losses and halted the training process prematurely. It then automatically restored the optimal weights from the best-performing epoch. These strategies ensured the final model’s excellent generalization on the independent test set.

H. Transfer Learning

Instead of training a custom architecture from scratch, we implemented a transfer learning approach using the MobileNetV2 network. The model was instantiated with weights pre-trained on the ImageNet dataset. Initially, the base convolutional layers were frozen to preserve the generalized visual feature extractors, including edge and texture detectors. Then, a custom classification head was appended and trained. This head comprised a GlobalAveragePooling2D layer, a dense layer with 256 units (and 0.5 dropout), and a final Softmax output for the 260 fruit classes. The transfer learning approach proved to be highly effective. The MobileNetV2 model converged rapidly, reaching a validation accuracy of over 95% by the second epoch. On the independent test set, it achieved an outstanding overall accuracy of 97%. More

importantly, the pre-trained model achieved the highest macro-averaged precision, recall, and F1 score of all the tested architectures, each reaching 0.98. Compared to the custom large CNN, MobileNetV2 uses its highly optimized depthwise separable convolutions to extract features more efficiently. This demonstrates that pretrained ImageNet weights provide a remarkably strong foundation, even for specialized fruit classification tasks.

I. Results and Validation

A total of seven experiments were conducted to evaluate the impact of different architectural choices on classifying the 260 fruit categories. Table II summarizes the performance metrics of all models on the independent test set. The baseline CNN model provided a solid foundation, achieving an overall accuracy of 94% and macro-averaged metrics of 0.94. However, modifying the architecture yielded significant improvements. The custom “Large CNN” model and the transfer learning approach using the pre-trained “MobileNetV2” model demonstrated the highest predictive capabilities, achieving a test accuracy of 97%. The Large CNN was the best custom-built model; however, MobileNetV2 slightly outperformed it in terms of macro-averaged precision, recall, and F1-score (0.98 vs. 0.97). This indicates MobileNetV2’s superior ability to distinguish between highly similar subcategories. Importantly, all evaluated models exceeded the project’s minimum requirements of 80% accuracy and 70% precision/recall by a large margin.

TABLE II
PERFORMANCE COMPARISON OF CNN ARCHITECTURES ON THE TEST SET.

Experiment	Test Acc.	Macro Precision	Macro Recall	Macro F1
Baseline CNN	0.94	0.94	0.94	0.94
CNN + Augmentation	0.95	0.96	0.95	0.95
Small CNN	0.97	0.97	0.96	0.96
Large CNN	0.97	0.97	0.97	0.97
AveragePooling CNN	0.94	0.95	0.94	0.94
Stride/Padding CNN	0.96	0.96	0.96	0.96
MobileNetV2	0.97	0.98	0.98	0.98

The Large CNN, the best custom architecture, was validated by analyzing specific individual predictions. Fig. 4 illustrates three best-case scenarios in which the model correctly predicted the fruit category with nearly 100% confidence. These successful classifications usually involve distinct fruits with unique shapes and colors that the network can easily distinguish from the standardized white background.



Fig. 4. Best-case scenarios: Three examples of highly confident and correct classifications.

On the other hand, an analysis of the misclassifications (Fig. 5) shows three worst-case scenarios in which the model made confident yet incorrect predictions, all involving different Blackberry varieties being mistaken for one another. Fig. 6 presents a focused confusion matrix for the 20 classes with the highest total error counts, which allows us to investigate this pattern systematically. Three of the four Blackberry varieties (Blackberry 1, 3, and 4) account for the highest error counts among all 260 classes, each with 145 to 150 misclassified images, compared to 21 to 80 for the other classes shown. Furthermore, unlike most other high-error classes, whose misclassifications are dispersed across the broader label space, the Blackberry varieties are predominantly confused with each other, as visible from the off-diagonal entries within this group in Fig. 6. This confirms that the model’s residual errors are largely systematic rather than random, occurring primarily where the interclass variance between visually near-identical subcategories of the same fruit is minimal, even though the model almost always identifies the correct broader fruit family.



Fig. 5. Worst-case scenarios: Three examples of incorrect classifications, mostly occurring between visually similar varieties.

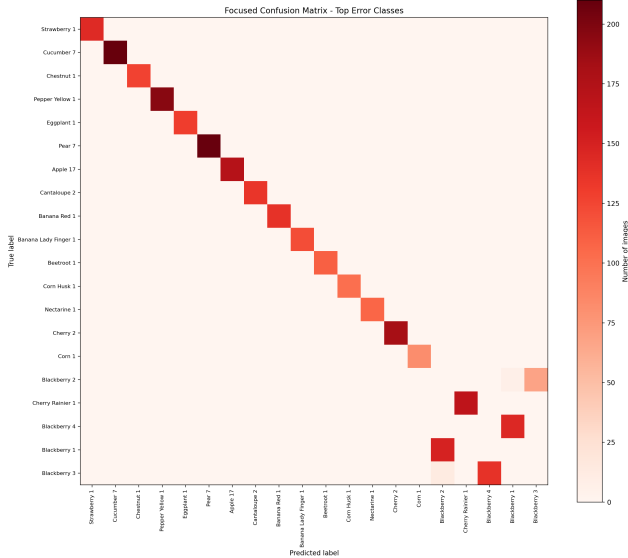


Fig. 6. Focused confusion matrix for the 20 classes with the highest total error counts (Large CNN).

III. RNN MODEL FOR TEMPERATURE FORECASTING

A. Dataset and Preprocessing

The Jena Climate dataset [8] contains 420,551 observations recorded every 10 minutes from January 2009 to January 2017 at the Max Planck Institute for Biogeochemistry in Jena, Germany. Each record includes a timestamp and 14 meteorological variables, summarized in Table III.

TABLE III
VARIABLES IN THE JENA CLIMATE DATASET.

Variable	Unit	Description
Date Time	—	Observation timestamp
p	mbar	Atmospheric pressure
T	°C	Air temperature
T_{pot}	K	Potential temperature
T_{dew}	°C	Dew point temperature
rh	%	Relative humidity
VP_{max}	mbar	Saturation vapor pressure
VP_{act}	mbar	Actual vapor pressure
VP_{def}	mbar	Vapor pressure deficit
sh	g/kg	Specific humidity
H_2O	mmol/mol	Water vapor concentration
ρ	g/m ³	Air density
w_v	m/s	Wind speed
max w_v	m/s	Maximum wind speed
wd	degrees	Wind direction

1) *Data Cleaning*: Wind speed and maximum wind speed contained 18 and 20 erroneous values of -9999 , which are physically impossible measurements; these were replaced with NaN.

2) *Indexing*: The timestamp column was parsed into date-time format and set as the index, enabling pandas to treat the data as a proper time series and allowing time-based operations such as chronological splitting.

3) *Feature Selection*: Features were selected based on the correlation matrix and meteorological knowledge. Variables that could be derived from temperature or were nearly identical to an already selected feature were dropped. The final feature set consists of air temperature T (°C), which serves as the target variable and whose historical values capture temporal dependencies; atmospheric pressure p , as pressure changes are often associated with changing weather systems that may influence temperature; dew point temperature T_{dew} , which is closely related to atmospheric moisture conditions and temperature; relative humidity rh, which provides information about the amount of moisture present in the air relative to its capacity; actual vapor pressure VP_{act} and vapor pressure deficit VP_{def} , which characterize moisture content and evaporation conditions in the atmosphere; wind speed w_v , which may provide information about the transport of air masses and local weather changes; and wind direction w_d , which can indicate the origin of incoming air masses associated with different temperature regimes.

4) *Data Preparation*: After the feature selection, rows containing NaN values were dropped. Then the rows containing wind direction and time were cyclically encoded to produce valuable results. Wind direction and time features

were cyclically encoded using sine and cosine transformations. Wind direction is circular by nature: without encoding, 0° and 360° would appear as opposite values despite representing the same direction. The same applies to time, where 23:50 and 00:00 are one step apart but numerically distant. The data was split chronologically into train, validation, and test sets using a 70/15/15 ratio, preserving temporal order to avoid training on future data. Min-max scaling was applied to normalize all features to the range $[0, 1]$, ensuring compatibility with the RMSLE evaluation metric which requires non-negative values. Finally, input sequences were constructed with a forecast horizon of 144 steps (24 hours) and an initial window size of 72 steps (12 hours).

B. Methodology

To isolate the effect of recurrent layer type and window size, all other hyperparameters were kept constant across experiments. The number of units was fixed at 64, as the experiments showed that it had better results than 32 across both architectures. A batch size of 256 was chosen as a balance between training speed and stable gradient estimates. Smaller batches introduce more noise into each update, while larger batches demand more memory without consistently improving generalisation [9]. The Adam optimizer was chosen for its efficiency, as it requires little tuning and works well on large datasets with many parameters [7]. The chosen loss function is MSE, which penalises larger errors more heavily, making the model less prone to large mistakes [1]. Early stopping was used with a maximum of 30 epochs, a patience of 5, monitoring validation loss and restoring the best weights to mitigate overfitting.

C. Evaluation Metric

The Root Mean Square Logarithmic Error (RMSLE) is defined as:

$$\text{RMSLE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\log(1 + \hat{y}_i) - \log(1 + y_i))^2} \quad (1)$$

Since temperature values in the dataset can be negative, a constant shift was added to both predicted and actual values before computing the metric, ensuring all inputs to the logarithm remain positive and the RMSLE is well-defined.

D. Impact of RNN Layers

TABLE IV
RMSLE RESULTS FOR ALL MODEL AND WINDOW SIZE COMBINATIONS.

Model	Window	RMSLE	Epochs Run
LSTM	24h	0.0902	8
LSTM	48h	0.0904	8
SimpleRNN	12h	0.0907	19
LSTM	12h	0.0909	11
SimpleRNN	24h	0.0913	12
SimpleRNN	48h	0.0932	8

Table IV summarises the results of both architectures across different window sizes. At the shortest window of 12 hours,

both architectures perform similarly, with LSTM converging faster (11 epochs versus 19 for SimpleRNN). As the window size increases, the performance gap widens: LSTM achieves its best RMSLE of 0.0902 at 24 hours, while SimpleRNN degrades progressively, reaching its worst result of 0.0932 at 48 hours. The largest performance difference between the two architectures occurs at the 48-hour window.

This behaviour is consistent with the known vanishing gradient problem in traditional RNNs [1]. As sequence length increases, gradients tend to diminish during backpropagation through time, limiting the model's ability to learn long-term dependencies. LSTM mitigates this through its gating mechanism, allowing it to selectively retain relevant information across longer horizons and remain robust as the window size increases.

E. Impact of Time Windows

To assess the effect of the lookback window on forecast performance, experiments were conducted with windows of 12, 24, and 48 hours, corresponding to 72, 144, and 288 time steps respectively. A 3-day window (432 steps) was excluded due to memory constraints.

For LSTM, performance improves from 12h (0.0909) to 24h (0.0902), then marginally worsens at 48h (0.0904). These results suggest that extending the context beyond 24 hours provides little additional predictive benefit, and may introduce redundant or noisy historical information that marginally harms generalisation. The LSTM's loss curves remain smooth across all window sizes, reflecting stable and robust training regardless of sequence length.

For SimpleRNN, performance degrades consistently with window size, from 0.0907 at 12h to 0.0932 at 48h, accompanied by more erratic validation loss curves. This indicates that longer windows not only hurt accuracy but also reduce training stability, confirming the architecture's inability to retain relevant information over extended sequences due to the vanishing gradient problem.

Considering all six combinations jointly, architecture choice has a more dominant and consistent effect on RMSLE than window size. LSTM outperforms SimpleRNN in five of the six configurations, and its advantage grows with sequence length. The LSTM with a 24h window achieves the best balance of accuracy, convergence speed, and robustness, making it the recommended combination for this forecasting task.

1) Best Model Architecture: The best-performing configuration was an LSTM network with a single recurrent layer of 64 units followed by a Dense output layer, as summarised in Table V and Fig. 7. Although the 48-hour window provided more historical information, it did not improve forecasting performance. The RMSLE increased slightly from 0.0902 to 0.0904, suggesting that using more than 24 hours of past data provided little benefit and may have added unnecessary information. Therefore, the 24-hour LSTM model was selected as the best configuration because it achieved the lowest error with a simple architecture.

TABLE V
MODEL SUMMARY OF THE BEST ARCHITECTURE (LSTM, 24H WINDOW).

Layer (Type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 64)	19,712
dense_3 (Dense)	(None, 1)	65
Total params		59,333
Trainable params		19,777
Non-trainable params		0
Optimizer params		39,556

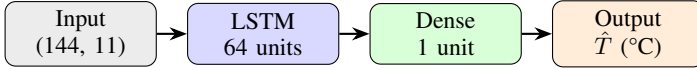


Fig. 7. Architecture of the best RNN model (LSTM, 24h window).

F. Results and Validation

Figs. 8 and 9 show the training and validation loss curves for each configuration. LSTM curves are consistently smoother across all window sizes, indicating stable convergence. SimpleRNN exhibits more erratic validation behaviour, particularly at the 12h window where a sharp spike occurs in later epochs, suggesting that the model struggles to generalise consistently even on shorter sequences. In both architectures, the gap between training and validation loss remains small, indicating no significant overfitting. Furthermore, all models achieved RMSLE values close to 0.09, demonstrating good forecasting accuracy.

Table VI presents the best and worst individual predictions of the LSTM 24h model on the test set. The three best predictions match the observed temperatures exactly to the reported precision, resulting in an absolute error of 0.0000°C. These cases demonstrate the model’s ability to capture stable temperature conditions with very high precision. In contrast, the three worst predictions occur at consecutive time steps (samples 31235-31237), where the model overestimates the temperature by approximately 12°C. The fact that these large errors occur consecutively suggests a sudden change in weather conditions that was not adequately captured by the 24-hour input window.

TABLE VI
BEST AND WORST CASE PREDICTIONS OF THE LSTM 24H MODEL.

Case	Sample	Actual (°C)	Predicted (°C)	Error (°C)
Best	42127	20.43	20.43	0.0000
Best	32540	17.48	17.48	0.0000
Best	38509	15.33	15.33	0.0000
Worst	31235	13.39	25.64	12.2515
Worst	31236	13.25	25.42	12.1738
Worst	31237	13.03	25.37	12.3444

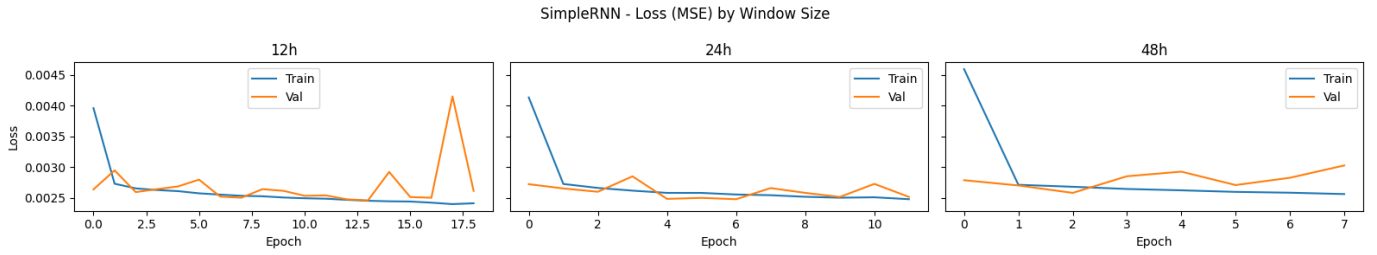


Fig. 8. SimpleRNN training and validation loss (MSE) across window sizes.

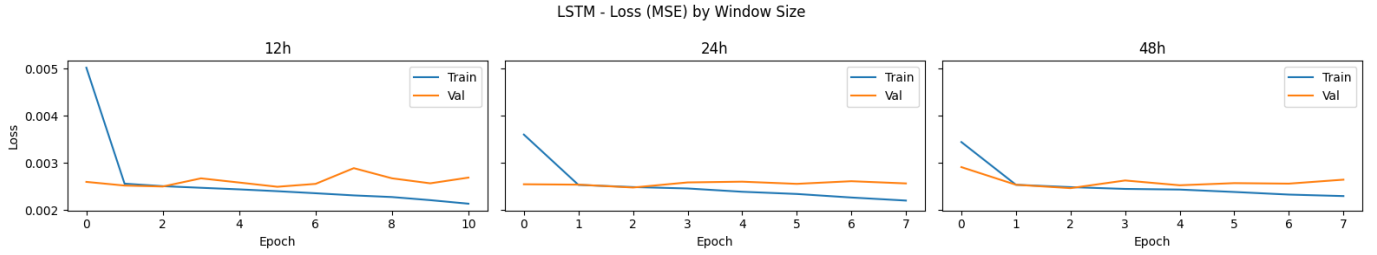


Fig. 9. LSTM training and validation loss (MSE) across window sizes.

IV. CONCLUSIONS

This work successfully developed and evaluated deep learning models for image classification and time series forecasting. In the fruit classification task, experiments showed that, although a lightweight CNN can achieve strong results due to the clean nature of the dataset, increasing the network's capacity alongside proper regularization provides the most robust custom solution. However, transfer learning via MobileNetV2 was the most efficient approach, converging rapidly and achieving the highest macro-averaged metrics. Architectural choices such as MaxPooling were significantly more effective than AveragePooling, while data augmentation offered only marginal improvements.

In the temperature forecasting task, six combinations of architecture and window size were evaluated. LSTM consistently outperformed SimpleRNN, particularly at longer windows, where the vanishing gradient problem caused SimpleRNN to degrade both in accuracy and training stability. The best configuration was an LSTM with a 24-hour lookback window, achieving an RMSLE of 0.0902. Extending the window beyond 24 hours provided no meaningful benefit, suggesting that the most recent 24 hours of meteorological data are sufficient for accurate next-day temperature forecasting. All models achieved RMSLE values well below the required threshold of 1, confirming the effectiveness of the LSTM architecture for this task.

Overall, both tasks demonstrated that careful architecture selection and hyperparameter tuning are critical in deep learning, and that simpler, well-tuned models can match or outperform more complex ones when the data is clean and well-structured.

ACKNOWLEDGMENT

The authors would like to acknowledge the use of DeepL Write and NotebookLM. These tools were utilized for structural text refinement, grammatical corrections, and readability improvements.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [2] "TensorFlow documentation." [Online]. Available: <https://www.tensorflow.org/overview>
- [3] A. Ananthakumar and A. Szysiak, "Deep learning – CNN and RNN practical work," 2026. [Online]. Available: <https://github.com/jagfoljersolen/deep-learning-cnn-rnn-isep>
- [4] "Fruits-360 dataset." [Online]. Available: <https://www.kaggle.com/moltean/fruits>
- [5] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2015.
- [6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Proc. ICLR*, 2015.
- [8] "Jena climate dataset." [Online]. Available: <https://www.kaggle.com/datasets/mnassrib/jena-climate>
- [9] N. S. Keskar *et al.*, "On large-batch training for deep learning," in *Proc. ICLR*, 2017.